

```

ITable expectedTable = expds.getTable("customer");
IDatabaseConnection connection = getConnection();
IDataSet databaseDataSet = connection.createDataSet();
ITable actualTable = databaseDataSet.getTable("customer");
Assertion.assertEquals(expectedTable, actualTable);

```

The test case implemented above uses the *DBTestCase* class. It will make DbUnit perform a *CLEAN_INSERT* operation before the test (i.e., delete all rows already present, and then perform a fresh insert of all the rows in the dataset), and no clean-up operation after it. If you want to change this behaviour, you can always override the *getSetUpOperation()* and *getTearDownOperation()* methods.

What we did in the example above is not always necessary; you can always use the normal JUnit TestCase class, implement your own *setUp()* and *tearDown()* methods, and perform the desired database operations manually. A skeleton of such a class should look like the following:

```

public class EmployeeSampleTest extends TestCase {

    public EmployeeSampleTest(String name)
    {
        super(name);
    }

    IDatabaseTester databaseTester;

    public void setUp() throws Exception{
        databaseTester = new JdbcDatabaseTester("com.mysql.
        jdbc.Driver", "jdbc:mysql://localhost/test", "root",
        "vampire");
        IDataSet dataSet = new FlatXmlDataSetBuilder().
        build(new FileInputStream("customer-init.xml"));
        databaseTester.setDataSet(dataSet);
        databaseTester.onSetup();
    }

    public void testInsert() {
        //your test method here
    }

    public void tearDown() throws Exception {
        databaseTester.onTearDown();
    }
}

```

As you can see, this method looks cleaner, and we do not need all those repeated *System.setProperty()* calls. Here we have used the *JdbcDatabaseTester*, while DbUnit also provides *DataSourceDatabaseTester* and *JndiDatabaseTester*. Also, *databaseTester.onSetup()* and *databaseTester.onTearDown()* create/close the database connection, and call the *getSetUpOperation()* and *getTearDownOperation()* methods as mentioned before.

Enhance your DbUnit code

In conclusion, here are a few pointers that will help make your DbUnit code better:

- While executing all the examples here, you might notice a warning about the Default Datatype factory. Although you can ignore it while working with the examples, if you use database-specific data-types while writing production code, you might want to configure a *DataTypeFactory* and *MetaDataHandler* for your particular database, or implement your own. For MySQL, this can be configured in the following manner:

```

connection.getConfig().setProperty(DatabaseConfig.
PROPERTY_DATATYPE_FACTORY, new
MySQLDataTypeFactory());
connection.getConfig().setProperty(DatabaseConfig.
PROPERTY_METADATA_HANDLER, new
MySQLMetadataHandler());

```

- You should always use a DTD for your XML datasets, because it can be beneficial in some cases, and a cure for many problems that may occur otherwise:

```

FlatXmlDataSet.write(connection.createDataSet(), new
FileOutputStream("employee.dtd"));

```

- Extending the *DBTestCase* class creates and closes the connection to the database for each test case. This can be quite a burden if you have a large number of test cases, so it's better to use the same connection for the whole test suite. This can be done by doing things manually rather than relying on DbUnit to manage connections for us, and also with the help of 'Before' and 'After' annotations, if you are using JUnit 4.
- DbUnit also provides an *AntTask* to be used with Ant, which allows controlling a database within an Ant build file.
- If there are one or more foreign keys in the dataset, we can get dependent tables with the help of the *TablesDependencyHelper* class, which will pull in all dependent tables into the flat XML dataset, in order of insertion. **END** 

Further reading

- [1] DbUnit website: www.dbunit.org
- [2] DbUnit made easy: <http://bit.ly/aXwRwT>
- [3] Effective Unit Testing with DbUnit: <http://bit.ly/drlrfl>

By: Ankit Mathur

The author is a 21-year old geek. He has a crush on Java, and also loves flirting with almost all other stuff related to Web technologies. Feel free to poke fun at this article, and direct your feedback to ankitreloaded@gmail.com.